

Collaboration & Git

Full Stack: Frontend, Backend, Database & Deployment · Class 4

What is a Project?

- A collection of files and folders that together make up something you're building
- Code, images, configuration files, documentation — all part of the project
- Today's "project" could just as easily be your collaborative story from earlier

What is a Repository (Repo)?

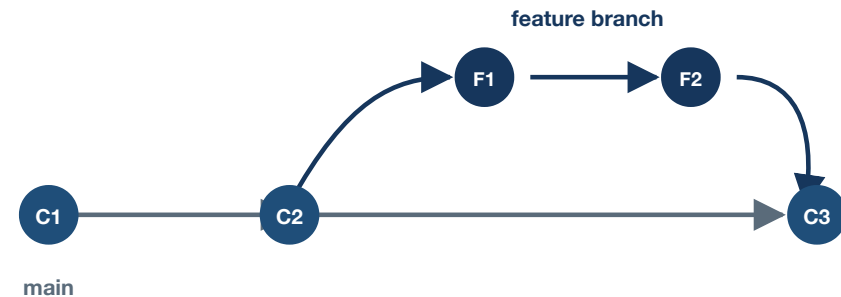
- A special folder that holds your project plus the entire history of every change made to it
- Git tracks who changed what, and when
- Think of it as a project folder with a built-in time machine

What is Git?

- A version control system — a tool that tracks changes to your files
- Lets you save snapshots ("commits") and go back to any previous version
- Runs entirely on your own computer — no internet required

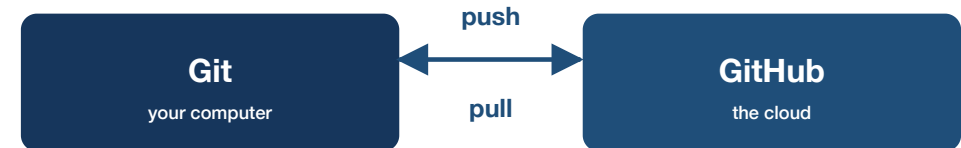
Git in Depth

- Distributed — every clone holds the entire project history
- Commits form a timeline, each one pointing back to its parent
- Branches let you build a feature without touching the main line
- Old commits stay reachable as long as something points to them



What is GitHub?

- A website that hosts Git repositories online
- Git is the tool; GitHub is the cloud service that stores your repositories
- Lets you access, share, and collaborate on a repo from anywhere
- Alternatives exist too — GitLab, Bitbucket



Git vs. GitHub

- Git — a tool installed on your computer; GitHub — a website that hosts your repositories
- Git — works completely offline; GitHub — needs an internet connection
- Git — tracks history through commits and branches; GitHub — adds collaboration on top: issues, pull requests, code review
- Git is the software everyone runs; GitHub is one company's hosting service built around it

Public vs. Private Repositories

- Public: anyone on the internet can see the code
- Private: only invited people can see or access it
- Team/company projects default to private until they're ready to be shared

Popular Open-Source Licenses

- A license spells out what others may legally do with your code
- MIT — very permissive: use, modify, or sell it, just keep the credit
- Apache 2.0 — like MIT, plus explicit patent protection
- GPL — free to use, but any modified version must stay open-source too

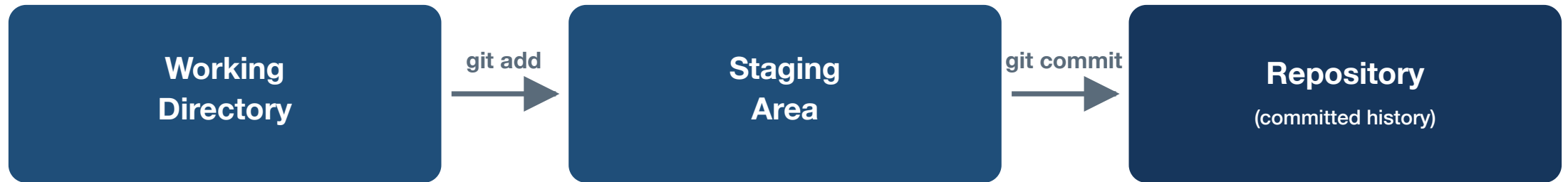
Collaborators & Invitations

- The repo owner adds teammates as collaborators by their GitHub username
- An invited member must accept before they can view or edit a private repo
- This is what keeps a private repository from being editable by just anyone

What is Cloning?

- Creates a complete copy of a repository from GitHub onto your own computer
- You get all the files and the entire project history, not just a snapshot
- Once cloned, you can work offline and sync back to GitHub later

The Three States of a File



`git status` — What Changed?

- Reports every file Git has noticed change since the last commit
- Distinguishes untracked (new), modified, and staged files
- The command to run before you add or commit anything

The Staging Area

- A middle ground between your working directory and the repository
- `git add` tells Git exactly which changes to include in the next commit
- Like packing a suitcase — you choose what goes in before you close it

What is a Commit?

- A snapshot of your project at one point in time
- Records what changed, who changed it, when, and a message describing why
- Like a save point in a video game — you can always return to it

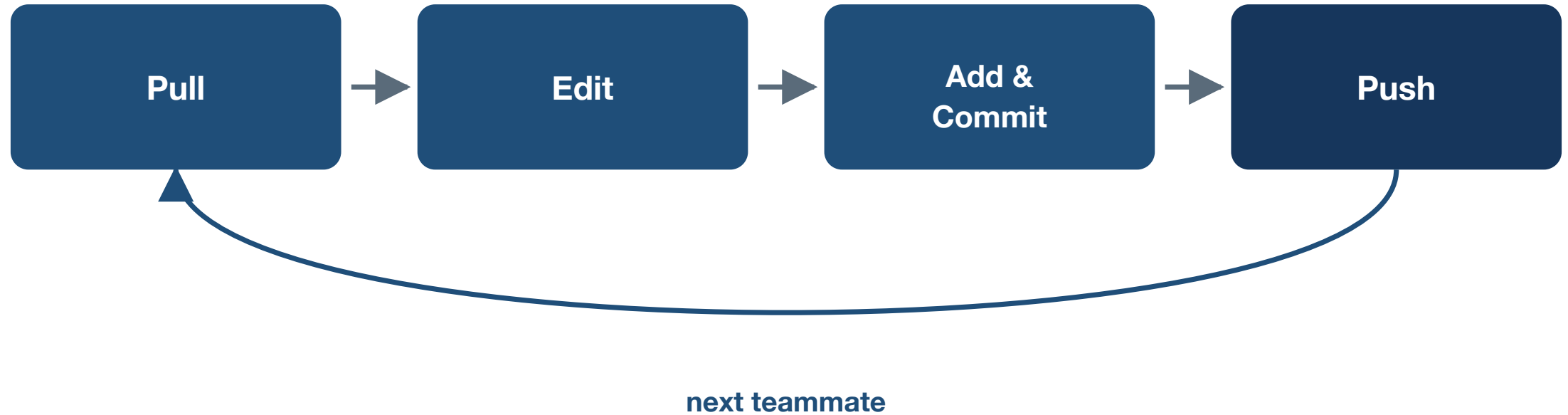
What is Push?

- Uploads your local commits to a remote repository like GitHub
- Until you push, your commits only exist on your own computer
- Backs your work up online and makes it visible to teammates

What is Pull?

- Downloads the latest changes from GitHub to your local computer
- Always pull before you start editing, so you're working on the latest version
- Prevents conflicts from two people changing the same thing differently

The Team Workflow



`git log` — The Commit History

- Lists every commit ever made to the repository, in order
- Each entry shows the author, timestamp, and commit message
- A full, permanent record of who contributed what, and when

Why Git Beats Manual Sharing

- No more emailing files back and forth or overwriting someone's work
- Every change is tracked, attributed, and reversible
- Scales from a 5-person story to codebases with thousands of contributors